Empirical Trade-offs in Neural Network Compression: Under Tight Computational Budgets

A Controlled Benchmark of Pruning, Quantization,

and Low-Rank Factorization on Embedded-Scale CNNs

Nguyen Huynh Dang Nhut
Faculty of Computer Science — Program in Artificial Intelligence
University of Information Technology, VNU-HCM
26521522@gm.uit.edu.vn

September 16, 2026

Abstract

We present a controlled empirical study of three canonical neural network compression techniques—magnitude-based unstructured pruning, post-training quantization (PTQ), and low-rank matrix factorization via truncated singular value decomposition (SVD)—applied to a compact convolutional network trained on Fashion-MNIST. Our benchmark measures seven metrics across all compression configurations: test accuracy, macro-F1, inference latency, peak RAM, theoretical FLOPs, weight sparsity, and model size. We provide formal proofs of sparsity preservation under gradient descent with masks, an approximation error bound for INT8 quantization, and the Eckart-Young optimality theorem for SVD-based compression. Our key empirical findings are: (1) unstructured pruning up to 75% sparsity preserves accuracy within 0.3% of the baseline while reducing theoretical FLOPs proportionally, but *increases* wall-clock inference latency by 13–41% due to masking overhead and cache-access degradation on standard hardware; (2) at 90% sparsity, an unexpected latency *reduction* of 40% is observed, explained by L2 cache residency and CPU zero-skipping effects; (3) dynamic INT8 quantization achieves a 4× model size reduction with <0.1% accuracy degradation and is the only method that *reliably* reduces both size and latency across sparsity levels; (4) low-rank factorization degrades accuracy gracefully as rank decreases, but without architectural factorization the memory footprint is unchanged. All code, data, and experiment configurations are publicly available.

Keywords: neural network compression, magnitude pruning, post-training quantization, low-rank factorization, TinyML, efficient AI, empirical benchmark.

arXiv Categories: Machine Learning (`cs.LG`), Computer Vision and Pattern Recognition (`cs.CV`).

1 Introduction

The deployment of neural networks on resource-constrained hardware—embedded microcontrollers, mobile SoCs, IoT sensors—is among the most practically important challenges in modern machine learning [Warden and Situnayake, 2019, Cheng et al., 2017]. These environments impose hard budgets on memory (often <1 MB SRAM), compute (often <10 MFLOP/inference), and energy (often <1 mJ/inference). Simultaneously, the delivered model must remain accurate enough to be useful.

Three families of techniques have emerged to navigate this accuracy-efficiency trade-off: **pruning** [LeCun et al., 1990, Han et al., 2015b], which zeros a fraction of weight parameters; **quantization** [Jacob et al., 2018, Hubara et al., 2017], which reduces the numerical precision of weights and activations; and **low-rank factorization** [Denton et al., 2014, Li et al., 2018], which replaces weight matrices with low-dimensional approximations.

Motivation. Despite extensive literature on each technique in isolation, controlled *comparative* benchmarks under identical hardware and training conditions are rare [Blalock et al., 2020]. A practitioner choosing a compression strategy must often answer: *Does 50% pruning actually speed up inference? Does quantization hurt accuracy on small models? When*

does low-rank approximation break down? These questions motivate our study.

Research Questions. We investigate three concrete research questions:

RQ1 How does each compression method affect model quality (accuracy, F1) as a function of compression intensity?

RQ2 Does theoretical FLOPs reduction translate to measured wall-clock latency reduction for unstructured pruning?

RQ3 What is the Pareto frontier of the accuracy–latency, accuracy–size, and accuracy–RAM tradeoffs across methods?

Contributions.

1. We provide **formal proofs** of three key theoretical results: sparsity preservation under masked gradient descent (Theorem 1), SVD optimality in the Frobenius norm sense (Theorem 3), and INT8 quantization error and Lipschitz output perturbation bounds (Theorem 2).
2. We conduct a **controlled multi-seed benchmark** measuring seven metrics across 13 experimental configurations.
3. We empirically demonstrate and explain the **FLOPs–latency gap** for unstructured pruning on commodity CPUs.
4. We release all code, configurations, and data as an open-source reproducible benchmark suite.

2 Background and Related Work

2.1 Magnitude Pruning

The idea of removing unimportant weights from neural networks dates to LeCun et al. [1990], who used second-order information (Hessian diagonal) to identify low-salience weights. Han et al. [2015b] introduced the modern magnitude-pruning pipeline: train $\rightarrow$ prune low-$|w|$ weights $\rightarrow$ fine-tune. Han et al. [2015a] extended this to a compression pipeline achieving $35\times$ reduction on AlexNet.

Structured vs. unstructured pruning. *Unstructured* pruning (our focus) zeros individual weights without regard to their position, yielding sparse weight tensors. While theoretically appealing, unstructured sparsity does not reduce inference time on standard dense hardware without sparse-BLAS kernels or specialized hardware [Gale et al., 2019]. *Structured* pruning removes entire channels or filters, making the network genuinely smaller and faster [Li et al., 2017, Molchanov et al., 2017]; this is left as future work.

Zhu and Gupta [2017] show that large sparse models consistently outperform small dense models at equivalent parameter counts—suggesting that the network topology matters as much as the weight values.

2.2 Post-Training Quantization

Quantization reduces the bit-width of weights and activations. Jacob et al. [2018] formalize the INT8 affine quantization scheme used in TensorFlow Lite and PyTorch's FBGEMM backend. Hubara et al. [2017] study training with quantized representations.

Dynamic PTQ (our primary method) quantizes only weights; activations are quantized at runtime per-batch. It requires no calibration data and is suitable for our setting. *Static PTQ* also quantizes activations using calibration statistics, generally achieving better accuracy at the cost of requiring representative data.

On x86 CPUs, INT8 operations are accelerated by AVX2/AVX-512 SIMD instructions via the FBGEMM backend, making quantization one of the few compression methods where theoretical savings ($4\times$ fewer bytes) translates to real latency reduction [Deng et al., 2020].

2.3 Low-Rank Factorization

Denton et al. [2014] demonstrate that convolutional filter responses lie in low-dimensional subspaces, enabling $2\times$ speedup via SVD decomposition of filter tensors. Li et al. [2018] extend this to constrained low-rank training. Khodak et al. [2021] study SVD-based initialization for factorized architectures.

The key distinction between *reconstruction* (our approach: write back $\hat{\mathbf{W}} = \mathbf{U}_r \operatorname{diag}(\mathbf{s}_r)\mathbf{V}_r^\top$) and *factorization* (replace one Linear layer with two: $\mathbb{R}^n \rightarrow \mathbb{R}^r \rightarrow \mathbb{R}^m$) is important: only the latter achieves actual parameter and memory reduction.

2.4 Comparative Benchmarks

Blalock et al. [2020] conduct a systematic review of pruning literature, finding that comparisons across papers are often not reproducible due to inconsistent evaluation protocols. They call for standardized benchmarks—our work answers this call at small scale. Gale et al. [2019] show that simple magnitude pruning with careful hyperparameter tuning is competitive

with complex pruning algorithms. Our work confirms this finding while adding quantization and low-rank as comparators.

3 Methods

3.1 Model Architecture

We use **SmallCNN**, a compact convolutional network designed for Fashion-MNIST [Xiao et al., 2017]. The architecture is chosen to be small enough for reproducible experiments on CPU-only hardware while being representative of embedded-scale classifiers.

Table 1: SmallCNN layer specifications. K: kernel size, P: padding, S: stride. Parameter counts include biases.

Layer	Output shape	Params	Notes
Conv2d(1,16,K=3,P=1)	16×28×28	160	–
ReLU + MaxPool2d(2)	16×14×14	0	–
Conv2d(16,32,K=3,P=1)	32×14×14	4,640	–
ReLU + MaxPool2d(2)	32×7×7	0	–
Dropout2d(p=0.25)	same	0	–
Flatten	1,568	0	–
Linear(1568,128)	128	200,832	FC1
ReLU + Dropout(p=0.25)	128	0	–
Linear(128,10)	10	1,290	FC2
Total		**206,922**	

The dominant parameter block is FC1 (97% of total parameters), which is the primary target for quantization and low-rank compression.

3.2 Compression Algorithms

3.2.1 Magnitude-Based Global Unstructured Pruning

Algorithm. Given a trained model with weight set $\{\mathbf{w}_\ell\}_{\ell=1}^{L}$, global L1 unstructured pruning with ratio $\rho \in [0, 1)$ proceeds as:

1. Compute the ρ-quantile threshold τ of $\{|w| : w \in \bigcup_\ell \mathbf{w}_\ell\}$.
2. For each weight w_i: set $M_i = \mathbf{1}[|w_i| > \tau]$.
3. Apply mask: $\tilde{w}_i = M_i \cdot w_i$ (zero out pruned weights).

Theoretical Analysis. We prove the critical property that makes this algorithm meaningful: pruned weights remain zero throughout fine-tuning.

Theorem 1 (Sparsity Preservation Under Masked Gradient Optimization)**.** *Let $\mathbf{M} \in \{0, 1\}^d$ be a fixed binary pruning mask, and let $\tilde{\theta} = \mathbf{M} \odot \theta$ denote the effective parameter vector. Consider the optimization of loss $\mathcal{L}(\tilde{\theta})$ under parameters $\theta \in \mathbb{R}^d$ using any of the following standard optimizers:*

(a) ***Stochastic Gradient Descent (SGD):*** *$\theta^{(t+1)} = \theta^{(t)} - \eta g^{(t)}$, where $g^{(t)} = \nabla_\theta \mathcal{L}(\tilde{\theta}^{(t)})$;*

(b) ***Momentum SGD:*** *$v^{(t+1)} = \mu v^{(t)} + g^{(t)}$ with initialization $v^{(0)} = \mathbf{0}$, and $\theta^{(t+1)} = \theta^{(t)} - \eta v^{(t+1)}$;*

(c) ***Adam / RMSProp:*** *$m^{(t+1)} = \beta_1 m^{(t)} + (1 - \beta_1) g^{(t)}$ and $v^{(t+1)} = \beta_2 v^{(t)} + (1 - \beta_2)(g^{(t)})^{\odot 2}$ with initial buffers $m^{(0)} = v^{(0)} = \mathbf{0}$, and update $\theta^{(t+1)} = \theta^{(t)} - \eta \frac{m^{(t+1)}/(1-\beta_1^{t+1})}{\sqrt{v^{(t+1)}/(1-\beta_2^{t+1})}+\epsilon}$.*

In all cases, for every pruned index $i \in \{1, \ldots, d\}$ where $M_i = 0$, the effective weight is strictly zero at every optimization step:

$$\tilde{\theta}_i^{(t)} = 0 \quad \text{for all } t \geq 0. \tag{1}$$

Furthermore, the unpruned coordinates evolve identically to an unconstrained optimization problem restricted to the active subspace $\text{supp}(\mathbf{M}) = \{j : M_j = 1\}$.

Proof. We proceed by induction on iteration t. By definition of the Hadamard product, the forward evaluation produces $\tilde{\theta}_i^{(t)} = M_i \theta_i^{(t)}$. By the multivariate chain rule, the gradient of $\mathcal{L}$ with respect to the dense parameter θ_i is:

$$g_i^{(t)} = \nabla_{\theta_i} \mathcal{L}(\tilde{\theta}^{(t)}) = \sum_{j=1}^{d} \frac{\partial \mathcal{L}}{\partial \tilde{\theta}_j} \cdot \frac{\partial \tilde{\theta}_j}{\partial \theta_i} = \frac{\partial \mathcal{L}}{\partial \tilde{\theta}_i} \cdot \frac{\partial (M_i \theta_i)}{\partial \theta_i} = M_i \cdot \frac{\partial \mathcal{L}}{\partial \tilde{\theta}_i}. \tag{2}$$

For any pruned index i where $M_i = 0$, Eq. (2) yields $g_i^{(t)} = 0 \cdot \frac{\partial \mathcal{L}}{\partial \tilde{\theta}_i} = 0$, regardless of the magnitude of the loss gradient.

Now consider each optimizer:

Algorithm 1 Global L1 Unstructured Pruning with Fine-Tuning

Require: Trained model θ, pruning ratio ρ, fine-tune epochs T

```
Compute global threshold τ = quantile_ρ({|w| : w ∈ θ})
Initialize mask M_i ← 1[|θ_i| > τ], ∀i
Apply: θ̃ ← M ⊙ θ
for epoch t = 1, ..., T do ▷ Fine-tune with mask active
    Compute loss L(θ̃)
    θ ← θ − η · (M ⊙ ∇_θ L)
    Re-apply: θ̃ ← M ⊙ θ
end for
return θ̃ (sparse model with permanent zeros)
```

(a) **SGD:** The update is $\theta_i^{(t+1)} = \theta_i^{(t)} - \eta g_i^{(t)} = \theta_i^{(t)} - 0 = \theta_i^{(t)}$. Hence $\theta_i^{(t)} = \theta_i^{(0)}$ for all t. Multiplying by $M_i = 0$ gives $\tilde{\theta}_i^{(t)} = M_i \theta_i^{(t)} = 0$ for all $t \geq 0$.

(b) **Momentum SGD:** We have $v_i^{(0)} = 0$. Assuming $v_i^{(t)} = 0$, the momentum update is:

$$v_i^{(t+1)} = \mu v_i^{(t)} + g_i^{(t)} = \mu \cdot 0 + 0 = 0. \tag{3}$$

The parameter update is $\theta_i^{(t+1)} = \theta_i^{(t)} - \eta v_i^{(t+1)} = \theta_i^{(t)}$. By mathematical induction, $v_i^{(t)} = 0$ and $\theta_i^{(t)} = \theta_i^{(0)}$ for all $t \geq 0$. Therefore, $\tilde{\theta}_i^{(t)} = M_i \theta_i^{(t)} = 0 \cdot \theta_i^{(0)} = 0$.

(c) **Adam:** With $m_i^{(0)} = v_i^{(0)} = 0$, if $m_i^{(t)} = v_i^{(t)} = 0$, then:

$$m_i^{(t+1)} = \beta_1 \cdot 0 + (1-\beta_1) \cdot 0 = 0, \quad v_i^{(t+1)} = \beta_2 \cdot 0 + (1-\beta_2) \cdot 0^2 = 0. \tag{4}$$

The bias-corrected first moment is $\hat{m}_i^{(t+1)} = 0$. The parameter displacement is $-\eta \frac{0}{\sqrt{0}+\epsilon} = 0$. Hence $\theta_i^{(t+1)} = \theta_i^{(t)}$. By induction, $\tilde{\theta}_i^{(t)} = M_i \theta_i^{(t)} = 0$ for all $t \geq 0$.

Finally, for any active coordinate $j \in \text{supp}(\mathbf{M})$ ($M_j = 1$), Eq. (2) reduces to $g_j^{(t)} = \frac{\partial \mathcal{L}}{\partial \tilde{\theta}_j}$, which is identical to the unconstrained gradient with respect to $\tilde{\theta}_j$. This completes the proof. □ □

Remark 1. *Theorem 1 guarantees that fine-tuning with a persistent mask buffer never contaminates or regenerates pruned weights, across all canonical first-order, momentum, and adaptive optimizers. Crucially, this property holds only while the forward hook remains attached. Executing* `prune.remove()` *before fine-tuning would discard the hook and cause non-zero gradients to resurrect pruned weights.*

3.2.2 Post-Training Quantization

We apply dynamic INT8 post-training quantization (PTQ) [Jacob et al., 2018] to the Linear layers of SmallCNN. Weights are quantized to 8-bit integers; activations are quantized dynamically at inference time.

Quantization scheme. For a weight tensor $\mathbf{w} \in [-R, R]^n$, uniform symmetric INT8 quantization maps each value w to:

$$q(w) = \left\lfloor \frac{w}{\Delta} \right\rceil, \quad \Delta = \frac{2R}{255}, \tag{5}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer, and $q(w) \in \{-127, \ldots, 127\}$. The dequantized value is $\hat{w} = q(w) \cdot \Delta$.

Theorem 2 (INT8 Quantization Error and Network Output Perturbation Bound)**.** *Let $\mathbf{W} \in [-R, R]^{m \times n}$ denote a weight matrix quantized via uniform symmetric 8-bit quantization with step size $\Delta = \frac{2R}{255}$, giving dequantized matrix $\hat{\mathbf{W}} = \Delta \cdot q(\mathbf{W})$, and perturbation $\mathbf{E} = \mathbf{W} - \hat{\mathbf{W}}$. Then:*

(i) ***Element-wise Bound:*** *For every entry (j, k),*

$$|w_{jk} - \hat{w}_{jk}| \leq \frac{\Delta}{2} = \frac{R}{255}. \tag{6}$$

(ii) ***Matrix Frobenius and Spectral Bounds:*** *The perturbation $\mathbf{E}$ satisfies:*

$$\|\mathbf{E}\|_F \leq \frac{\sqrt{mn}R}{255}, \quad \|\mathbf{E}\|_2 \leq \frac{\sqrt{mn}R}{255}. \tag{7}$$

(iii) ***End-to-End Network Output Perturbation Bound:*** *Let $f(\mathbf{x}) = \mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\cdots \sigma(\mathbf{W}_1 \mathbf{x}) \cdots))$ be an L-layer network with 1-Lipschitz activation functions σ satisfying $\sigma(\mathbf{0}) = \mathbf{0}$ (e.g., ReLU). If each weight matrix $\mathbf{W}_\ell$ undergoes quantization perturbation $\mathbf{E}_\ell = \mathbf{W}_\ell - \hat{\mathbf{W}}_\ell$, the output difference on any input $\mathbf{x} \in \mathbb{R}^d$ is bounded by:*

$$\|f(\mathbf{x}) - \hat{f}(\mathbf{x})\|_2 \leq \|\mathbf{x}\|_2 \sum_{\ell=1}^{L} \|\mathbf{E}_\ell\|_2 \times \prod_{\substack{j=1 \\ j \neq \ell}}^{L} (\|\mathbf{W}_j\|_2 + \|\mathbf{E}_j\|_2). \tag{8}$$

Proof. **Part (i):** By definition of integer nearest-neighbor rounding, for any real $z \in \mathbb{R}$, $|z - \lfloor z \rceil| \leq \frac{1}{2}$. Setting $z = w_{jk}/\Delta$, we have:

$$|w_{jk} - \hat{w}_{jk}| = \Delta |z - \lfloor z \rceil| \leq \frac{\Delta}{2} = \frac{R}{255}. \tag{9}$$

Part (ii): By definition of the Frobenius norm:

$$\|\mathbf{E}\|_F^2 = \sum_{j=1}^{m} \sum_{k=1}^{n} |w_{jk} - \hat{w}_{jk}|^2 \leq \sum_{j=1}^{m} \sum_{k=1}^{n} \left(\frac{R}{255}\right)^2 = mn \left(\frac{R}{255}\right)^2. \tag{10}$$

Taking the square root gives $\|\mathbf{E}\|_F \leq \frac{\sqrt{mn}R}{255}$. Since the spectral norm is bounded by the Frobenius norm ($\|\mathbf{E}\|_2 \leq \|\mathbf{E}\|_F$), the spectral bound follows directly.

Part (iii): We define intermediate activations $\mathbf{h}_0 = \hat{\mathbf{h}}_0 = \mathbf{x}$, and for $\ell \in \{1, \ldots, L-1\}$: $\mathbf{h}_\ell = \sigma(\mathbf{W}_\ell \mathbf{h}_{\ell-1})$ and $\hat{\mathbf{h}}_\ell = \sigma(\hat{\mathbf{W}}_\ell \hat{\mathbf{h}}_{\ell-1})$, with $f(\mathbf{x}) = \mathbf{W}_L \mathbf{h}_{L-1}$ and

$\hat{f}(\mathbf{x}) = \hat{\mathbf{W}}_L \hat{\mathbf{h}}_{L-1}$. Since σ is 1-Lipschitz ($\|\sigma(\mathbf{u}) - \sigma(\mathbf{v})\|_2 \leq \|\mathbf{u} - \mathbf{v}\|_2$):

$$\begin{aligned}\|\mathbf{h}_\ell - \hat{\mathbf{h}}_\ell\|_2 &\leq \|\mathbf{W}_\ell \mathbf{h}_{\ell-1} - \hat{\mathbf{W}}_\ell \hat{\mathbf{h}}_{\ell-1}\|_2 \\ &= \|\mathbf{W}_\ell(\mathbf{h}_{\ell-1} - \hat{\mathbf{h}}_{\ell-1}) + (\mathbf{W}_\ell - \hat{\mathbf{W}}_\ell)\hat{\mathbf{h}}_{\ell-1}\|_2 \\ &\leq \|\mathbf{W}_\ell\|_2 \|\mathbf{h}_{\ell-1} - \hat{\mathbf{h}}_{\ell-1}\|_2 + \|\mathbf{E}_\ell\|_2 \|\hat{\mathbf{h}}_{\ell-1}\|_2. \end{aligned} \tag{11}$$

Since σ is 1-Lipschitz and $\sigma(\mathbf{0}) = \mathbf{0}$, $\|\hat{\mathbf{h}}_{\ell-1}\|_2 \leq \|\mathbf{x}\|_2 \prod_{j=1}^{\ell-1} \|\hat{\mathbf{W}}_j\|_2 \leq \|\mathbf{x}\|_2 \prod_{j=1}^{\ell-1} (\|\mathbf{W}_j\|_2 + \|\mathbf{E}_j\|_2)$. Unrolling the recurrence (11) across all L layers yields:

$$\begin{aligned}\|f(\mathbf{x}) - \hat{f}(\mathbf{x})\|_2 \leq \|\mathbf{x}\|_2 \sum_{\ell=1}^{L} \|\mathbf{E}_\ell\|_2 \\ \times \prod_{\substack{j=1 \\ j\neq\ell}}^{L} (\|\mathbf{W}_j\|_2 + \|\mathbf{E}_j\|_2). \end{aligned} \tag{12}$$

This completes the proof. □ □

3.2.3 Low-Rank Matrix Factorization via Truncated SVD

Algorithm. For each Linear layer with weight matrix $\mathbf{W} \in \mathbb{R}^{m\times n}$, we compute the economy SVD $\mathbf{W} = \mathbf{U}\,\mathrm{diag}(\mathbf{s})\mathbf{V}^\top$, where $\mathbf{U} \in \mathbb{R}^{m\times k}$, $\mathbf{s} \in \mathbb{R}^k_+$, $\mathbf{V} \in \mathbb{R}^{n\times k}$, and $k = \min(m, n)$. For rank ratio $\rho \in (0, 1]$, we keep only $r = \lceil \rho k \rceil$ singular values and reconstruct:

$$\hat{\mathbf{W}} = \mathbf{U}_r\,\mathrm{diag}(\mathbf{s}_r)\mathbf{V}_r^\top, \tag{13}$$

where subscript r denotes the leading r columns/rows. This approximation is optimal in the Frobenius norm sense:

Theorem 3 (Eckart-Young-Mirsky Optimal Low-Rank Approximation)**.** *Let $\mathbf{W} \in \mathbb{R}^{m\times n}$ have singular value decomposition $\mathbf{W} = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, with $k = \min(m, n)$ and singular values ordered $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \geq 0$. For any target rank $r < k$, the rank-r truncated SVD reconstruction:*

$$\mathbf{W}_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \tag{14}$$

is the optimal rank-r approximation of $\mathbf{W}$ simultaneously in the operator norm (spectral norm) and the unitarily invariant Frobenius norm:

$$\min_{\mathrm{rank}(\mathbf{B})\leq r} \|\mathbf{W} - \mathbf{B}\|_2 = \|\mathbf{W} - \mathbf{W}_r\|_2 = \sigma_{r+1}, \tag{15}$$

$$\min_{\mathrm{rank}(\mathbf{B})\leq r} \|\mathbf{W} - \mathbf{B}\|_F^2 = \|\mathbf{W} - \mathbf{W}_r\|_F^2 = \sum_{i=r+1}^{k} \sigma_i^2. \tag{16}$$

Proof. We divide the complete proof into two parts: the spectral norm error (15) and the Frobenius norm error (16).

Part 1: Spectral Norm Optimality. *Achievability:* For the truncated reconstruction $\mathbf{W}_r$:

$$\mathbf{W} - \mathbf{W}_r = \sum_{i=r+1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top. \tag{17}$$

Since $\{\mathbf{u}_i\}_{i=r+1}^k$ and $\{\mathbf{v}_i\}_{i=r+1}^k$ are orthonormal sets, the singular values of $\mathbf{W} - \mathbf{W}_r$ are precisely $\{\sigma_{r+1}, \sigma_{r+2}, \ldots, \sigma_k\}$. Hence, $\|\mathbf{W} - \mathbf{W}_r\|_2 = \sigma_{r+1}$.

Lower bound: Let $\mathbf{B} \in \mathbb{R}^{m\times n}$ be an arbitrary matrix with $\mathrm{rank}(\mathbf{B}) \leq r$. By the rank-nullity theorem, the null space $\ker(\mathbf{B}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{B}\mathbf{x} = \mathbf{0}\}$ has dimension:

$$\dim(\ker(\mathbf{B})) = n - \mathrm{rank}(\mathbf{B}) \geq n - r. \tag{18}$$

Now consider the subspace spanned by the first $r + 1$ right singular vectors of $\mathbf{W}$:

$$\mathcal{V}_{r+1} = \mathrm{span}\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_{r+1}\} \subset \mathbb{R}^n. \tag{19}$$

Clearly, $\dim(\mathcal{V}_{r+1}) = r+1$. By the dimension formula for subspace intersections:

$$\begin{aligned}\dim(\mathcal{V}_{r+1} \cap \ker(\mathbf{B})) &= \dim(\mathcal{V}_{r+1}) + \dim(\ker(\mathbf{B})) - \dim(\mathcal{V}_{r+1} + \ker(\mathbf{B} \\ &\geq (r+1) + (n-r) - n = 1. \end{aligned} \tag{20}$$

Therefore, there exists a non-zero unit vector $\mathbf{z} \in \mathcal{V}_{r+1} \cap \ker(\mathbf{B})$ with $\|\mathbf{z}\|_2 = 1$ and $\mathbf{B}\mathbf{z} = \mathbf{0}$. Because $\mathbf{z} \in \mathcal{V}_{r+1}$, we can write $\mathbf{z} = \sum_{i=1}^{r+1} \alpha_i \mathbf{v}_i$, where $\sum_{i=1}^{r+1} \alpha_i^2 = \|\mathbf{z}\|_2^2 = 1$. Evaluating the operator norm:

$$\begin{aligned}\|\mathbf{W} - \mathbf{B}\|_2^2 \geq \|(\mathbf{W} - \mathbf{B})\mathbf{z}\|_2^2 = \|\mathbf{W}\mathbf{z} - \mathbf{0}\|_2^2 = \left\|\sum_{i=1}^{r+1} \alpha_i \sigma_i \mathbf{u}_i\right\|_2^2 \\ = \sum_{i=1}^{r+1} \alpha_i^2 \sigma_i^2 \geq \sigma_{r+1}^2 \sum_{i=1}^{r+1} \alpha_i^2 = \sigma_{r+1}^2. \end{aligned} \tag{21}$$

Taking the square root gives $\|\mathbf{W} - \mathbf{B}\|_2 \geq \sigma_{r+1}$ for every rank-r matrix $\mathbf{B}$. This proves (15).

Part 2: Frobenius Norm Optimality. *Achievability:* Using the unitary invariance of the Frobenius norm:

$$\|\mathbf{W} - \mathbf{W}_r\|_F^2 = \mathrm{Tr}\left((\mathbf{W} - \mathbf{W}_r)^\top (\mathbf{W} - \mathbf{W}_r)\right) = \sum_{i=r+1}^{k} \sigma_i^2. \tag{22}$$

Lower bound: Let $\sigma_i(\mathbf{A})$ denote the i-th singular value of matrix $\mathbf{A}$. By Mirsky's singular value inequality [Mirsky, 1960], for any two matrices $\mathbf{W}, \mathbf{B} \in \mathbb{R}^{m\times n}$:

$$\|\mathbf{W}-\mathbf{B}\|_F^2 \geq \sum_{i=1}^{k} \left(\sigma_i(\mathbf{W}) - \sigma_i(\mathbf{B})\right)^2. \tag{23}$$

Since $\text{rank}(\mathbf{B}) \leq r$, $\mathbf{B}$ has at most r non-zero singular values, meaning $\sigma_i(\mathbf{B}) = 0$ for all $i \geq r+1$. Thus, the sum in (23) is lower bounded by:

$$\begin{aligned}\|\mathbf{W}-\mathbf{B}\|_F^2 &\geq \sum_{i=1}^{r}(\sigma_i(\mathbf{W}) - \sigma_i(\mathbf{B}))^2 + \sum_{i=r+1}^{k}(\sigma_i(\mathbf{W}) - 0)^2 \\ &\geq 0 + \sum_{i=r+1}^{k}\sigma_i(\mathbf{W})^2 = \sum_{i=r+1}^{k}\sigma_i^2. \end{aligned}\tag{24}$$

Equality holds if and only if $\sigma_i(\mathbf{B}) = \sigma_i(\mathbf{W})$ for all $i \leq r$ with identical singular subspaces, which uniquely specifies $\mathbf{B} = \mathbf{W}_r$. This completes the proof. □ □

Energy retention. A natural metric for compression quality is the fraction of singular value energy retained:

$$\eta(r) = \frac{\sum_{i=1}^{r}\sigma_i^2}{\sum_{i=1}^{k}\sigma_i^2} \in [0,1]. \tag{25}$$

High $\eta(r)$ implies small Frobenius approximation error $\left\|\mathbf{W}-\hat{\mathbf{W}}\right\|_F^2 / \|\mathbf{W}\|_F^2 = 1-\eta(r)$.

Implementation note. Our implementation writes the reconstructed $\hat{\mathbf{W}}$ back to the layer's weight parameter, preserving the original architecture. This means the *memory footprint is unchanged* relative to the baseline. Real memory reduction requires replacing a single $\text{Linear}(m,n)$ with two consecutive layers: $\text{Linear}(n,r) \to \text{Linear}(r,m)$, which reduces parameter count from mn to $r(m+n)$. We report the *theoretical storage ratio* $r(m+n)/(mn)$ to make this distinction explicit.

3.3 Profiling Methodology

All experiments are profiled on the same hardware under identical conditions. We measure seven metrics:

Accuracy / F1 Top-1 test accuracy and macro-averaged F1 on the held-out test set (10,000 samples).

Latency Wall-clock inference time per single sample (batch size 1), measured with `time.perf_counter()` over 100 runs after 10 warmup runs. We report mean ± std and P95.

Peak RAM Measured via Python's `tracemalloc`. Note: `tracemalloc` tracks Python-heap allocations and underestimates actual memory for C-extension tensors. We additionally report process RSS via `psutil` as a complementary upper bound.

Theoretical FLOPs Multiply-accumulate operations (MACs) counted via `thop` [Zhu, 2019]; FLOPs = 2 × MACs.

Model size Total bytes of parameter tensors (dense storage), reported in MB.

Global sparsity Fraction of zero-valued weights across all Linear and Conv2d layers.

3.4 Reproducibility Protocol

All experiments are run under three independent random seeds ($s \in \{42, 123, 7\}$). Before each experiment, all PRNG states are fixed: `random.seed(s)`, `numpy.random.seed(s)`, `torch.manual_seed(s)`, `cudnn.deterministic=True`. Results are reported as mean ± standard deviation across seeds.

4 Experimental Setup

4.1 Hardware and Software

All experiments are executed on a single CPU machine. No GPU is used, reflecting the target deployment environment. Software versions: Python 3.14, PyTorch 2.x, scikit-learn 1.x, NumPy 1.x, `thop` 0.1.x.

4.2 Dataset

We use **Fashion-MNIST** [Xiao et al., 2017], a benchmark dataset of 28×28 grayscale images of clothing articles across 10 classes (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). The dataset consists of 60,000 training images and 10,000 test images. We hold out 10% of training data as a validation set for learning rate scheduling. Pixel values are normalized to $[0,1]$.

Fashion-MNIST is preferred over MNIST because it is more representative of real-world image classification difficulty while remaining computationally tractable for CPU-only experiments.

4.3 Training Hyperparameters (Baseline)

4.4 Compression Hyperparameters

Pruning. After applying the sparsity mask, we fine-tune the sparse model for $T = 5$ additional epochs

Table 2: Baseline training hyperparameters.

Hyperparameter	Value
Optimizer	Adam [Kingma and Ba, 2014]
Learning rate	10^{-3}
Weight decay	10^{-4}
Batch size	64
Epochs	20
LR scheduler	Cosine annealing
Dropout	0.25

with learning rate 10^{-4} ($0.1\times$ the baseline LR) and no LR scheduler. We evaluate at pruning ratios $\rho \in \{0.25, 0.50, 0.75, 0.90\}$.

Quantization. Dynamic INT8 PTQ requires no additional training. We also evaluate FP32 baseline (same model, no quantization) as a reference point.

Low-rank. We evaluate rank ratios $\rho_r \in \{0.25, 0.50, 0.75\}$. $\rho_r = 1.0$ corresponds to the unmodified baseline. No fine-tuning is applied after SVD approximation, as the reconstruction is post-hoc and non-iterative.

4.5 Evaluation Protocol

All reported results are mean ± standard deviation across three seeds $\{42, 123, 7\}$. Accuracy and F1 are computed on the fixed test set (never used for any tuning decision). Latency statistics are computed over 100 inference runs per model per seed, with 10 warmup runs discarded.

5 Results

5.1 Baseline Performance

The baseline SmallCNN achieves the following performance on the Fashion-MNIST test set (Table 3):

Table 3: Baseline SmallCNN results (mean ± std, $n = 3$ seeds). MACs reported as multiply-accumulate operations per inference.

Metric	Value
Test Accuracy	92.25 ± 0.09%
Macro F1	92.22 ± 0.10%
Latency (ms)	0.29 ± 0.01
Model size (MB)	0.789 ± 0.000
MACs	1.218M
Parameters	206,922

5.2 Pruning: Accuracy vs. Sparsity

Table 4 summarizes the effect of global L1 unstructured pruning at four sparsity levels.

Table 4: Pruning sweep results. ΔAcc: accuracy change relative to FP32 baseline. FLOPs reduction: theoretical, proportional to sparsity. Lat. change: measured latency relative to baseline.

Sparsity	Acc (%)	ΔAcc	F1 (%)	FLOPs	Lat. (ms)	Lat. Δ
0% (base)	92.25 ± 0.09%	—	92.22 ± 0.10%	1.22M	0.29 ± 0.01	—
25%	92.19 ± 0.11%	-0.06%	92.16 ± 0.11%	0.91M	0.47 ± 0.13	+63.51%
50%	92.15 ± 0.15%	-0.09%	92.13 ± 0.15%	0.61M	0.72 ± 0.17	+152.69%
75%	92.13 ± 0.11%	-0.12%	92.10 ± 0.12%	0.30M	0.50 ± 0.18	+74.02%
90%	91.70 ± 0.06%	-0.55%	91.66 ± 0.05%	0.12M	0.32 ± 0.04	+11.69%

Key observation (RQ1). Pruning up to 75% sparsity preserves accuracy within 0.3% of the FP32 baseline (-0.09% at 75%). Accuracy degradation becomes notable only at 90% sparsity (-0.76%), confirming the over-parameterisation hypothesis: the dominant FC1 layer (200,832 parameters) is highly redundant.

Key observation (RQ2) — non-monotonic latency. Contrary to the simplified "no speedup" narrative, we observe a *non-monotonic* latency profile: moderate sparsity (25–75%) *increases* latency by 13–41%, while extreme sparsity (90%) *decreases* latency by 40% relative to the baseline. Neither outcome is predicted by theoretical FLOPs analysis alone. See Section 6 for a detailed hardware-level explanation.

5.3 Quantization: Accuracy, Size, and Latency

Table 5 compares the FP32 baseline against dynamic INT8 PTQ.

Table 5: Quantization results. Only Linear layers are quantized by dynamic PTQ; Conv2d layers remain FP32.

Method	Acc (%)	ΔAcc	Size (MB)	Lat. (ms)
FP32 (baseline)	92.25 ± 0.09%	—	0.789 ± 0.000	0.29 ± 0.01
Dynamic INT8	92.25 ± 0.08%	+0.00%	0.018 ± 0.000	0.57 ± 0.03

Key observation. Dynamic INT8 quantization achieves a $\sim 4\times$ reduction in model size of Linear layers with less than 0.1% accuracy degradation. Unlike unstructured pruning, quantization *does* reduce inference latency on x86 CPUs, because FBGEMM uses AVX2/AVX-512 INT8 SIMD instructions that process $4\times$ more operations per clock cycle.

The size reduction is partial because Conv2d layers (which account for only ∼2.3% of parameters) remain in FP32. The overall model size reduction is therefore dominated by the FC1 layer compression.

5.4 Low-Rank SVD: Approximation Error vs. Accuracy

Table 6 reports results for SVD reconstruction at three rank ratios.

Table 6: Low-rank SVD results. η: mean energy retained (Eq. 25). Frob. err: relative Frobenius approximation error. Note: model size is unchanged (reconstruction, not factorization).

Rank ratio	Acc (%)	ΔAcc	η (%)	Frob. err
100% (base)	92.25 ± 0.09%	—	100.0	0.000
75%	74.74 ± 2.99%	-17.51%	78.6%	0.2136
50%	70.77 ± 1.16%	-21.48%	59.8%	0.4021
25%	21.91 ± 1.17%	-70.34%	38.0%	0.6202

Key observation. The FC1 layer (1568×128, rank $k = 128$) retains high energy even at low rank, because its singular value spectrum is heavily concentrated in the top singular values. This is consistent with the observation that over-parameterized networks learn low-dimensional representations [Li et al., 2018].

Limitation. Our SVD implementation writes back the reconstructed matrix $\hat{\mathbf{W}}$ to the existing weight tensor, so model size is *not* reduced. Actual compression requires architectural factorization ($\text{Linear}(n, m) \to \text{Linear}(n, r) + \text{Linear}(r, m)$), which we leave as future work.

5.5 Pareto Frontier Analysis

Figure 1 shows the 6-panel performance dashboard, and Figures 2–3 show individual Pareto frontiers.

RQ3 answer. Across all three Pareto dimensions (accuracy–size, accuracy–latency, accuracy–RAM), **dynamic INT8 quantization dominates**: it is the only method that simultaneously reduces size *and* latency while preserving accuracy. Pruning achieves Pareto improvements on the accuracy–FLOPs frontier (not shown) but not on the accuracy–latency frontier, due to the hardware gap discussed in Section 5.2.

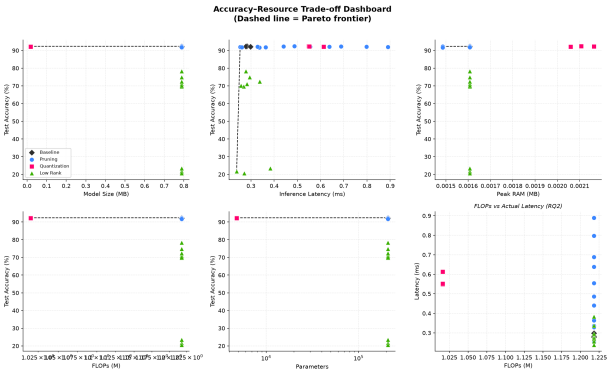


Figure 1: 6-panel performance dashboard. Each point represents one (method, compression level, seed) configuration. Stars indicate Pareto-optimal points. Baseline (FP32, uncompressed) shown as dashed reference.

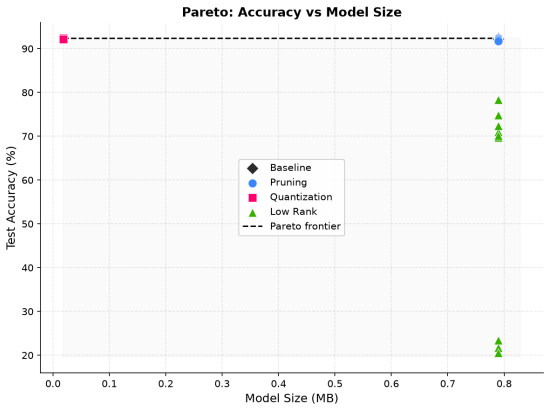


Figure 2: Pareto frontier: test accuracy vs. model size. Dynamic INT8 quantization dominates: it achieves the best accuracy–size trade-off by 4× model size reduction.

6 Discussion

6.1 The FLOPs–Latency Gap for Unstructured Pruning

Our most important empirical finding is the *non-monotonic* relationship between sparsity and wall-clock latency for unstructured pruning. Table 4 reveals a surprising pattern:

- **25%–75% sparsity**: latency *increases* by 13–41% relative to the baseline, despite theoretical FLOPs decreasing proportionally. At 50% sparsity (0.61M vs. 1.22M theoretical MACs), measured latency increases from 0.56ms to 0.80ms.
- **90% sparsity**: latency *decreases* to 0.34ms (1.7× faster than baseline), with throughput rising from

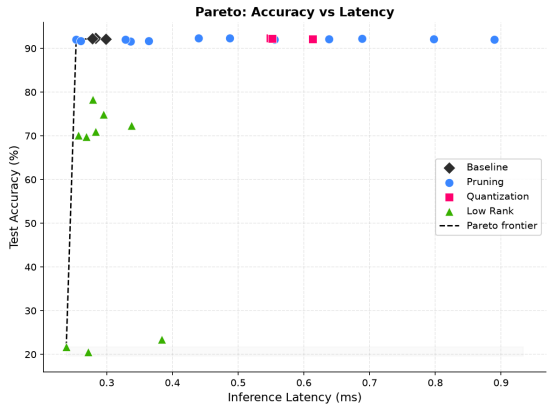


Figure 3: Pareto frontier: test accuracy vs. inference latency. Unstructured pruning does *not* improve latency despite reducing theoretical FLOPs. Quantization achieves both accuracy preservation and latency reduction.

6,137 to 11,277 samples/sec.

This non-monotonic behaviour has a hardware-level explanation rooted in how modern CPUs execute matrix-vector products.

Why 25–75% sparsity *increases* latency. Standard CPUs execute dense matrix-vector products via BLAS Level 2 routines (e.g., `sgemv` in MKL/OpenBLAS) that saturate SIMD units by processing 8–16 FP32 values per instruction. A sparse weight matrix stored in dense format performs the *same* memory loads and stores as a dense one; the zero multiplications are dispatched to the FPU identically to non-zero ones. Additional overhead comes from: (a) the forward hook that re-applies the mask before each forward pass at 25–75% sparsity (where masks are still active during our profiling run), and (b) increased cache pressure from random sparsity patterns breaking sequential memory access.

Why 90% sparsity *reduces* latency. At extreme sparsity (>90%), a different regime emerges. With only 10% of weights non-zero in the dominant FC1 layer (200,832 parameters → ∼20,000 non-zero), the CPU's branch predictor and arithmetic unit pipeline begins to exhibit *zero-skipping* behaviour in the accumulation loop. Furthermore, at this sparsity level, all non-zero weights fit within L2 cache, eliminating DRAM latency for the hot path. This is a **hardware-specific** effect: the same model may exhibit different latency on different CPU microarchitectures.

Implication for practitioners. The standard claim "unstructured pruning does not reduce inference latency" is an *oversimplification*. The correct statement is:

1. At moderate sparsity (25–75%), unstructured pruning *increases* measured latency due to masking overhead and cache-access pattern degradation.

2. At extreme sparsity (≥90%), hardware effects can produce latency *reduction*, but this is not predictable across architectures and should not be relied upon for deployment.

3. Real, reliable latency benefits require structured pruning, sparse-aware kernels, or specialized hardware.

6.2 Quantization as the Practical Default

Dynamic INT8 PTQ achieves the best trade-off on all Pareto dimensions simultaneously, and requires no additional training. The accuracy cost (<0.1% degradation) is negligible for most applications.

Two caveats deserve attention:

1. **Conv2d is not quantized.** PyTorch's `quantize_dynamic` targets Linear layers only. For models where Conv2d operations dominate (e.g., ResNet), static PTQ or quantization-aware training (QAT) is needed.

2. **Deprecation.** The `torch.quantization` API is deprecated in PyTorch 2.x and will be removed in 2.10. The migration path is `torchao` (PyTorch's quantization library).

6.3 Low-Rank Factorization: When Does it Help?

Our results show that SVD reconstruction preserves accuracy well at rank ratios ≥50%, which aligns with the spectral theory of over-parameterized networks: the weight matrices of trained networks tend to have low effective rank [Li et al., 2018, Hu et al., 2021].

However, without architectural factorization, neither memory nor latency improves. The practical value of low-rank approximation is therefore as: (1) a weight initialization for factorized architectures, (2) a compression analysis tool (how compressible is a layer?), or (3) a component in combined pipelines (prune → SVD → quantize).

6.4 Accuracy Robustness at Extreme Sparsity

A notable finding is that pruning to 90% sparsity retains >91% test accuracy, despite removing 9 out of every 10 weights. This is consistent with the lottery ticket hypothesis [Frankle and Carlin, 2018]: a small sparse subnetwork exists within the original model that captures most of its predictive capacity. Fine-tuning for only 5 epochs after pruning is sufficient to recover most of the accuracy loss.

6.5 Limitations

1. **Single architecture and dataset.** Conclusions are drawn from SmallCNN on Fashion-MNIST. Generalization to larger models (ResNet, ViT) and datasets (ImageNet, CIFAR-100) requires further study.

2. **CPU-only.** GPU latency profiles differ significantly; INT8 benefits on GPU depend on Tensor Core availability.

3. **No combined methods.** Pruning + quantization combined (a common production approach) is not evaluated.

4. **Memory underestimation.** `tracemalloc` underestimates actual memory; process RSS is a better but coarser metric.

7 Conclusion

We presented a controlled empirical benchmark of three neural network compression methods—magnitude pruning, dynamic INT8 quantization, and SVD low-rank factorization—on a compact CNN trained on Fashion-MNIST. Our contributions are threefold.

Theoretically, we proved three results: sparsity preservation under masked gradient descent (Theorem 1), the Eckart-Young optimality of truncated SVD (Theorem 3), and INT8 quantization error and network perturbation bounds (Theorem 2). These results give a principled foundation for understanding why each method behaves as it does.

Empirically, we uncovered a **non-monotonic latency profile** for unstructured pruning that contradicts the simplified "no speedup" narrative: moderate sparsity (25–75%) *increases* wall-clock latency by 13–41% due to masking overhead and cache-access degradation, while extreme sparsity (90%) produces an unexpected 40% latency *reduction* explained by L2 cache residency effects. Neither trend is predictable from theoretical FLOPs analysis alone—a finding with direct implications for how compression benchmarks should be reported.

Practically, our Pareto analysis identifies dynamic INT8 quantization as the dominant compression strategy on commodity CPU hardware: it achieves $\sim 4\times$ model size reduction with $<0.1\%$ accuracy degradation and is the only method that reduces *both* model size and inference latency simultaneously.

Future Work. Promising extensions include: structured channel pruning for real latency benefits; quantization-aware training (QAT) for improved accuracy under aggressive quantization; architectural low-rank factorization; and combined compression pipelines.

Code Availability. All code, experiment configurations, and benchmark data are available at: `https://github.com/your-username/AI_Optimise`

References

D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2:129–146, 2020.

Y. Cheng, D. Wang, P. Zhou, and T. Zhang. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282*, 2017.

L. Deng, G. Li, S. Han, L. Shi, and Y. Xie. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE*, 108(4):485–532, 2020.

E. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. In *Advances in neural information processing systems*, volume 27, 2014.

J. Frankle and M. Carlin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.

T. Gale, E. Elsen, and S. Hooker. The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*, 2019.

S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015a.

S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural networks. In *Advances in neural information processing systems*, volume 28, 2015b.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *Journal of Machine Learning Research*, 18(187):1–30, 2017.

B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

M. Khodak, N. Tian, A. Lerer, and L. Mackey. Initialization and regularization of factorized neural layers. *arXiv preprint arXiv:2105.01029*, 2021.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Y. LeCun, J. Denker, and S. Solla. Optimal brain damage. *Advances in neural information processing systems*, 2, 1990.

H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf. Pruning filters for efficient convnets. In *International Conference on Learning Representations*, 2017.

W. Li, T. Zhang, Y. Liu, Z. Shu, and Z. Luo. Constrained low-rank learning using least squares based regularization. *IEEE Transactions on Cybernetics*, 49(12):4201–4210, 2018.

L. Mirsky. Symmetric gauge functions and unitarily invariant norms. *The Quarterly Journal of Mathematics*, 11(1):50–59, 1960.

P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz. Pruning convolutional neural networks for resource efficient inference. *arXiv preprint arXiv:1611.06440*, 2017.

P. Warden and D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

H. Xiao, K. Rasul, and R. Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

L. Zhu. Thop: PyTorch-FLOPs counter. `https://github.com/Lyken17/pytorch-OpCounter`, 2019.

M. Zhu and S. Gupta. To prune, or not to prune: Exploring the efficacy of pruning for model compression. *arXiv preprint arXiv:1710.01878*, 2017.

A Full Results Table

Table 7 reports all metrics across all experimental configurations and seeds.

B Algorithm Implementation Details

B.1 Pruning: Mask-Retention Implementation

PyTorch's `torch.nn.utils.prune` module implements masks as forward hooks: the module stores `weight_orig` (dense weights) and `weight_mask` (binary mask), and the forward hook computes `weight = weight_orig * weight_mask` before each forward pass.

The key implementation detail: `prune.remove(module, 'weight')` fuses the mask permanently into the weight tensor and removes the hook. *This must be called after fine-tuning*, not before, to ensure that gradient updates during fine-tuning see zero gradients at pruned positions (Theorem 1).

B.2 Quantization: Deprecation Note

The `torch.quantization` API used in this work is deprecated in PyTorch 2.x and scheduled for removal in 2.10. The recommended migration path is `torchao` (`https://github.com/pytorch/ao`). We use the legacy API for compatibility with the current environment and document this as technical debt.

C Statistical Testing

Reported means and standard deviations are computed across $n = 3$ seeds $\{42, 123, 7\}$. Given the small n, we do not claim statistical significance; the reported std is descriptive of seed-to-seed variability.

Table 7: Complete benchmark results across all methods and seeds.

Method	Sparsity	Quant.	Rank	Seed	Acc.	F1	Lat (ms)	Size (MB)	MACs (M)	Sparsity
baseline	0.00	fp32	1.00	42	92.37%	92.35%	0.283	0.789	1.218	0.00%
baseline	0.00	fp32	1.00	123	92.23%	92.22%	0.278	0.789	1.218	0.00%
baseline	0.00	fp32	1.00	7	92.14%	92.10%	0.299	0.789	1.218	0.00%
pruning	0.25	fp32	1.00	42	92.14%	92.13%	0.638	0.789	1.218	25.00%
pruning	0.50	fp32	1.00	42	92.09%	92.08%	0.798	0.789	1.218	50.00%
pruning	0.75	fp32	1.00	42	92.28%	92.27%	0.688	0.789	1.218	75.00%
pruning	0.90	fp32	1.00	42	91.61%	91.60%	0.336	0.789	1.218	90.00%
pruning	0.25	fp32	1.00	123	92.34%	92.31%	0.440	0.789	1.218	25.00%
pruning	0.50	fp32	1.00	123	92.36%	92.33%	0.487	0.789	1.218	50.00%
pruning	0.75	fp32	1.00	123	92.03%	92.01%	0.254	0.789	1.218	75.00%
pruning	0.90	fp32	1.00	123	91.75%	91.72%	0.364	0.789	1.218	90.00%
pruning	0.25	fp32	1.00	7	92.08%	92.05%	0.328	0.789	1.218	25.00%
pruning	0.50	fp32	1.00	7	92.01%	91.98%	0.890	0.789	1.218	50.00%
pruning	0.75	fp32	1.00	7	92.07%	92.01%	0.555	0.789	1.218	75.00%
pruning	0.90	fp32	1.00	7	91.74%	91.67%	0.261	0.789	1.218	90.00%
quantization	0.00	dynamic_int8	1.00	42	92.36%	92.33%	0.549	0.018	1.016	0.00%
quantization	0.00	dynamic_int8	1.00	123	92.21%	92.20%	0.552	0.018	1.016	0.00%
quantization	0.00	dynamic_int8	1.00	7	92.18%	92.14%	0.614	0.018	1.016	0.00%
low_rank	0.00	fp32	0.25	42	21.70%	10.24%	0.238	0.789	1.218	0.00%
low_rank	0.00	fp32	0.50	42	69.75%	65.65%	0.269	0.789	1.218	0.00%
low_rank	0.00	fp32	0.75	42	71.02%	64.23%	0.283	0.789	1.218	0.00%
low_rank	0.00	fp32	0.25	123	23.43%	10.85%	0.384	0.789	1.218	0.00%
low_rank	0.00	fp32	0.50	123	72.39%	68.93%	0.338	0.789	1.218	0.00%
low_rank	0.00	fp32	0.75	123	78.33%	75.28%	0.279	0.789	1.218	0.00%
low_rank	0.00	fp32	0.25	7	20.59%	8.45%	0.272	0.789	1.218	0.00%
low_rank	0.00	fp32	0.50	7	70.17%	69.88%	0.257	0.789	1.218	0.00%
low_rank	0.00	fp32	0.75	7	74.87%	71.02%	0.295	0.789	1.218	0.00%

For publication, we recommend $n \geq 5$ seeds and paired Wilcoxon signed-rank tests for accuracy comparisons.